DOCS TUGAS 4

PEMROGRAMAN MOBILE



Implementasi Future, Asynchronous Programming, HTTP API, JSON Parsing, Navigasi, dan Error Handling

Disusun Oleh

VRAMUDHIA EGY PRATAMA

20240801382

Dosen Pengampu

JEFRY SUNUPURWA ASRI, S.Kom., M.Kom.

Fakultas Ilmu Komputer

Program Studi Teknik Informatika

BAB I – PENDAHULUAN

Laporan ini disusun berdasarkan dokumentasi praktikum pada project Flutter bernama praktikum_4. Materi yang terlihat pada dokumentasi mencakup penggunaan Future dan asynchronous programming, pengambilan data pengguna dari HTTP API, pengubahan data JSON menjadi objek Dart melalui model Pengguna, navigasi dari daftar ke halaman detail, serta penanganan error ketika endpoint atau koneksi jaringan mengalami masalah. Dokumentasi juga memperlihatkan halaman detail postingan yang memiliki komentar, sehingga data yang diproses tidak hanya berupa data pengguna tetapi juga data yang saling berhubungan.

Tujuan praktikum ini adalah memahami alur pengembangan aplikasi Flutter yang berkomunikasi dengan sumber data eksternal. Proses dimulai dari pembuatan tampilan sederhana dan fungsi asynchronous, kemudian dilanjutkan dengan pengambilan data melalui API, parsing JSON, penyajian data dalam daftar, navigasi menuju detail, serta pengujian kondisi berhasil dan gagal. Dengan mengikuti alur tersebut, mahasiswa dapat memahami bahwa aplikasi mobile tidak hanya membutuhkan tampilan antarmuka, tetapi juga membutuhkan pengelolaan proses asynchronous, struktur data, navigasi, dan penanganan kesalahan.

BAB II – PERSIAPAN DAN ALUR KERJA

Tahap awal dilakukan pada project Flutter praktikum_4 dengan file utama main.dart. Dari screenshot pertama terlihat project dibuka pada lingkungan pengembangan dan aplikasi dijalankan pada emulator Android. Struktur aplikasi menggunakan MaterialApp dan halaman utama SalamPage. Pada tahap ini fokus praktikum adalah memastikan project dapat dijalankan dengan baik sebelum menambahkan proses yang berhubungan dengan API.

Karena dokumentasi memperlihatkan penggunaan package http dan dart:convert, maka aplikasi membutuhkan dukungan HTTP untuk komunikasi dengan API dan dukungan JSON untuk mengubah respons API menjadi struktur data yang dapat diproses oleh Dart. Persiapan package dilakukan pada project Flutter sebelum kode API dijalankan.

```
flutter pub add http
```

Setelah dependency tersedia, proses pengembangan dapat dilakukan secara bertahap. Tahap pertama adalah menguji Future sederhana. Tahap kedua adalah mengambil daftar pengguna dari endpoint API. Tahap ketiga adalah membuat model Pengguna dan melakukan parsing JSON. Tahap keempat adalah menampilkan data pada ListTile dan menyediakan navigasi ke halaman detail. Tahap berikutnya adalah menguji kondisi error, termasuk endpoint yang salah dan koneksi yang gagal. Tahap terakhir adalah mengembangkan tampilan detail postingan beserta komentar.

BAB III – PANDUAN IMPLEMENTASI DARI AWAL

3.1 Menyiapkan Project Flutter

Buat atau buka project Flutter dengan nama praktikum_4, kemudian pastikan file lib/main.dart tersedia. Jalankan aplikasi pada emulator Android yang telah tersedia. Pada dokumentasi, project berhasil dijalankan dan emulator menampilkan aplikasi dengan judul Demo Future. Tahap ini penting karena seluruh implementasi berikutnya dilakukan pada project yang sudah dapat di-build dan dijalankan.

```
flutter run
```

Setelah aplikasi berhasil dijalankan, periksa tampilan awal sebelum menambahkan fitur berikutnya. Tujuannya adalah memastikan masalah yang muncul nantinya berasal dari perubahan kode yang sedang dikerjakan, bukan dari konfigurasi dasar project.

3.2 Menerapkan Future dan Asynchronous Programming

Pada tahap awal dibuat fungsi ambilSalam() yang bersifat asynchronous. Fungsi tersebut mengembalikan Future<String> dan menggunakan await Future.delayed() selama dua detik sebelum menghasilkan teks. Konsep ini memperlihatkan bahwa proses yang membutuhkan waktu dapat dijalankan secara asynchronous sehingga aplikasi tidak perlu menghentikan keseluruhan proses UI selama menunggu hasil.

```
Future<String> ambilSalam() async {
  await Future.delayed(const Duration(seconds: 2));
  return "Halo dari masa depan!";
}
```

Setelah fungsi dibuat, fungsi tersebut digunakan pada halaman SalamPage sehingga emulator menampilkan teks Halo dari masa depan!. Jika teks muncul setelah proses delay, berarti fungsi asynchronous berjalan sesuai alur yang diharapkan. Dokumentasi Foto 1 memperlihatkan hasil tersebut secara langsung.

3.3 Menyiapkan Komunikasi dengan HTTP API

Tahap berikutnya adalah mengambil data pengguna dari API. Dokumentasi menunjukkan penggunaan package http untuk mengirim request dan dart:convert untuk mengolah JSON. Endpoint yang digunakan pada kondisi berhasil adalah https://jsonplaceholder.typicode.com/users. Data dari endpoint tersebut kemudian diproses menjadi daftar objek pengguna.

```
final response = await http.get(
  Uri.parse("https://jsonplaceholder.typicode.com/users"),
);
```

Respons dari server perlu diperiksa sebelum diproses. Dokumentasi menunjukkan adanya pengecekan response.statusCode != 200. Jika status yang diterima bukan 200, aplikasi menghasilkan exception dengan informasi kegagalan. Pemeriksaan ini membuat aplikasi tidak langsung mencoba membaca data ketika server sebenarnya memberikan respons error.

3.4 Membuat Model Pengguna dan Parsing JSON

Agar data JSON lebih mudah digunakan, data pengguna direpresentasikan dalam sebuah model bernama Pengguna. Dokumentasi menunjukkan factory Pengguna.fromJson(Map<String, dynamic> json). Factory tersebut mengambil nilai berdasarkan key JSON seperti id, name, username, email, phone, website, dan city. Hasil akhirnya adalah objek Pengguna yang memiliki struktur data yang jelas.

```
factory Pengguna.fromJson(Map<String, dynamic> json) {
  // mengambil id, name, username, email,
  // phone, website, dan city dari JSON
}
```

Pendekatan model seperti ini membuat kode lebih terorganisasi. Data mentah dari API tidak digunakan secara langsung di semua bagian UI, melainkan terlebih dahulu diubah

menjadi objek Pengguna. Setelah menjadi objek, data dapat dikirim ke halaman lain melalui parameter konstruktor sehingga proses navigasi menjadi lebih terstruktur.

3.5 Menampilkan Daftar Pengguna

Setelah data API berhasil diubah menjadi objek Pengguna, data ditampilkan dalam halaman Daftar Pengguna. Dokumentasi memperlihatkan setiap pengguna menggunakan ListTile, avatar berupa huruf awal nama, nama pengguna, alamat email, dan ikon panah di sisi kanan. ListTile digunakan sebagai elemen daftar yang dapat dipilih sehingga pengguna aplikasi dapat berpindah dari daftar menuju informasi yang lebih lengkap.

Pada tahap ini, pengambilan data dan tampilan UI saling terhubung. Ketika daftar objek Pengguna telah tersedia, aplikasi melakukan iterasi terhadap data tersebut dan membuat elemen daftar secara dinamis. Artinya, jumlah item yang ditampilkan mengikuti jumlah data yang diterima dari API, bukan ditulis satu per satu secara manual.

3.6 Membuat Navigasi ke Detail Pengguna

Ketika salah satu pengguna pada halaman Daftar Pengguna dipilih, aplikasi menggunakan Navigator.push() dan MaterialPageRoute untuk berpindah ke DetailPenggunaPage. Objek Pengguna yang dipilih ikut dikirim ke halaman detail. Pada halaman tersebut ditampilkan informasi seperti email, nomor telepon, dan website. Dengan cara ini, halaman detail dapat menggunakan data yang sudah diperoleh sebelumnya tanpa harus melakukan request API baru untuk data yang sama.

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => DetailPenggunaPage(pengguna: pengguna),
  ),
);
```

3.7 Melakukan Pengujian Error pada API

Dokumentasi menunjukkan pengujian menggunakan endpoint yang salah, yaitu https://jsonplaceholder.typicode.com/user. Karena endpoint tersebut tidak ditemukan, aplikasi memperoleh status HTTP 404 dan menampilkan pesan Gagal memuat data (kode 404).

Pengujian ini penting untuk memastikan aplikasi tidak hanya diuji ketika kondisi normal, tetapi juga ketika server memberikan respons yang tidak sesuai.

Setelah endpoint diperbaiki menjadi https://jsonplaceholder.typicode.com/users, dokumentasi memperlihatkan kondisi error yang berbeda, yaitu ClientException dengan SocketException dan Connection refused. Kondisi tersebut menunjukkan bahwa masalah tidak lagi berupa endpoint yang salah, tetapi koneksi menuju server tidak berhasil. Kode juga menggunakan timeout selama 10 detik sehingga proses request memiliki batas waktu dan tidak menunggu tanpa batas.

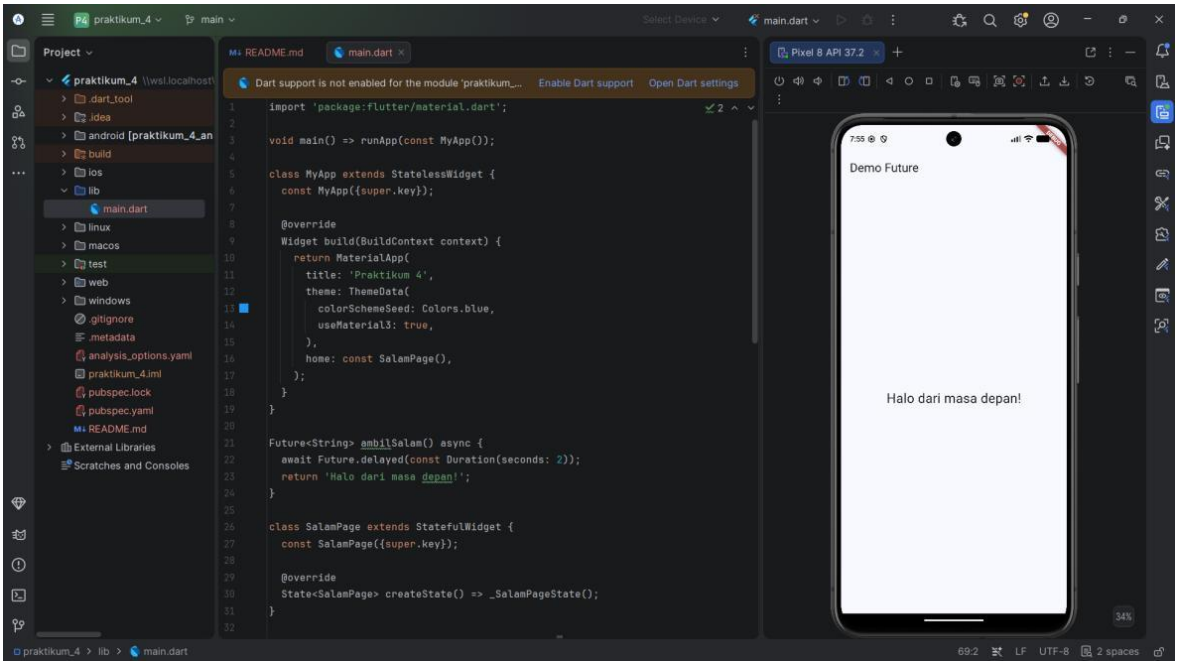
3.8 Menampilkan Detail Postingan dan Komentar

Tahap terakhir yang terdokumentasi adalah halaman Detail Postingan. Halaman ini menampilkan Post ID, User ID, judul postingan, dan isi postingan. Di bawah informasi utama terdapat bagian Komentar yang menampilkan nama dan email komentar. Tampilan ini memperlihatkan bahwa aplikasi dapat menangani data yang memiliki hubungan antara postingan, pengguna, dan komentar.

Pada bagian UI terlihat penggunaan Text() untuk menampilkan informasi teks dan SizedBox untuk mengatur jarak antar komponen. Dengan demikian, implementasi bukan hanya berfokus pada pengambilan data, tetapi juga pada penyusunan data agar dapat dibaca dengan jelas oleh pengguna aplikasi.

BAB IV – ANALISIS SETIAP FOTO

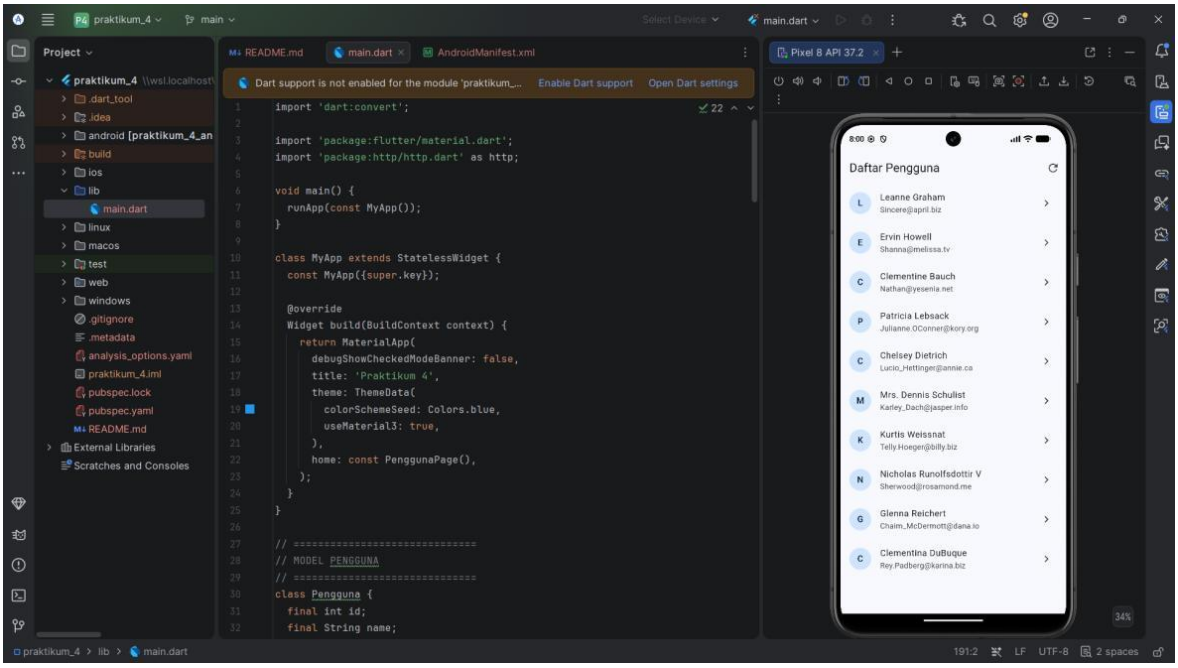
Foto 1 – Tampilan awal aplikasi dan penerapan Future



Gambar 1. Dokumentasi hasil praktikum – Tampilan awal aplikasi dan penerapan Future

Foto pertama memperlihatkan project Flutter praktikum_4 yang dibuka pada lingkungan pengembangan melalui file main.dart dan dijalankan pada emulator Android. Pada kode terdapat fungsi ambilSalam() yang menggunakan Future dan asynchronous programming. Fungsi tersebut memberikan jeda selama dua detik dengan Future.delayed() sebelum mengembalikan teks “Halo dari masa depan!”. Pada emulator terlihat judul “Demo Future” dan teks “Halo dari masa depan!” di bagian tengah layar. Hasil tersebut menunjukkan bahwa fungsi asynchronous berhasil dipanggil dan hasilnya dapat ditampilkan pada antarmuka aplikasi. Foto ini menjadi tahap dasar sebelum aplikasi dikembangkan untuk melakukan komunikasi dengan API.

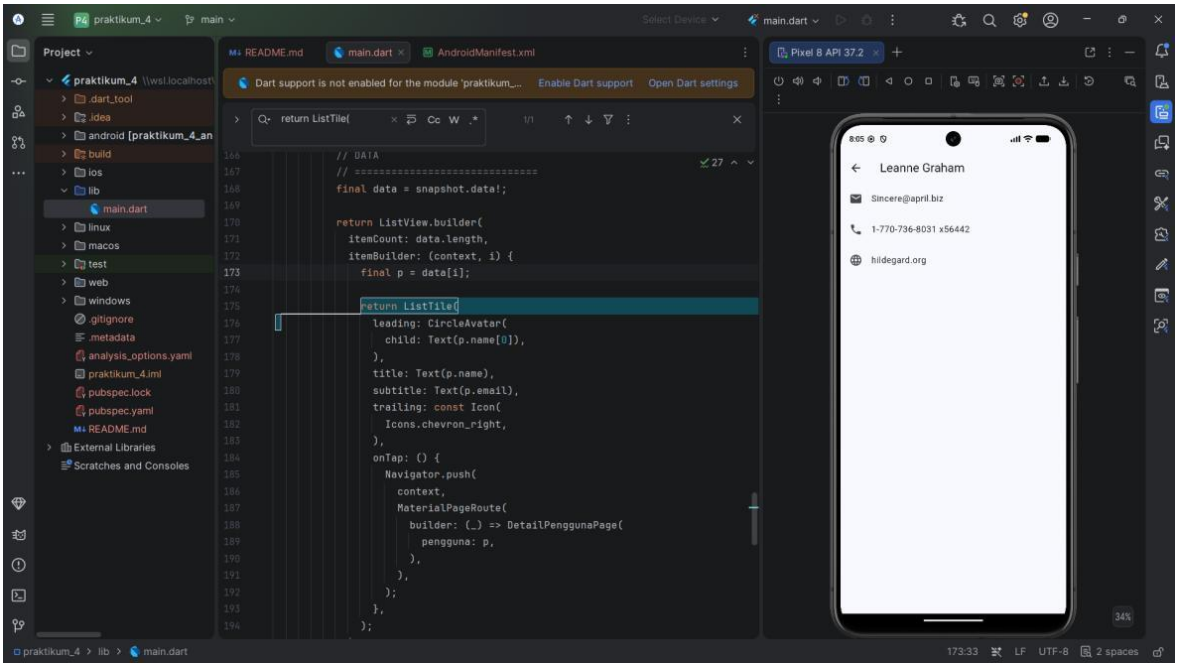
Foto 2 – Menampilkan daftar pengguna dari API



Gambar 2. Dokumentasi hasil praktikum – Menampilkan daftar pengguna dari API

Foto kedua memperlihatkan halaman "Daftar Pengguna" yang menampilkan data pengguna secara dinamis. Beberapa nama yang terlihat adalah Leanne Graham, Ervin Howell, Clementine Bauch, dan Patricia Lebsack, masing-masing disertai alamat email. Setiap item menggunakan ListTile dengan avatar berupa huruf awal nama dan ikon panah di sisi kanan. Dari sisi implementasi, package http digunakan untuk melakukan komunikasi dengan API, sedangkan dart:convert digunakan untuk mengolah respons JSON. Dengan demikian, foto ini menunjukkan hubungan antara proses request API, parsing data, dan penyajian data dalam bentuk daftar.

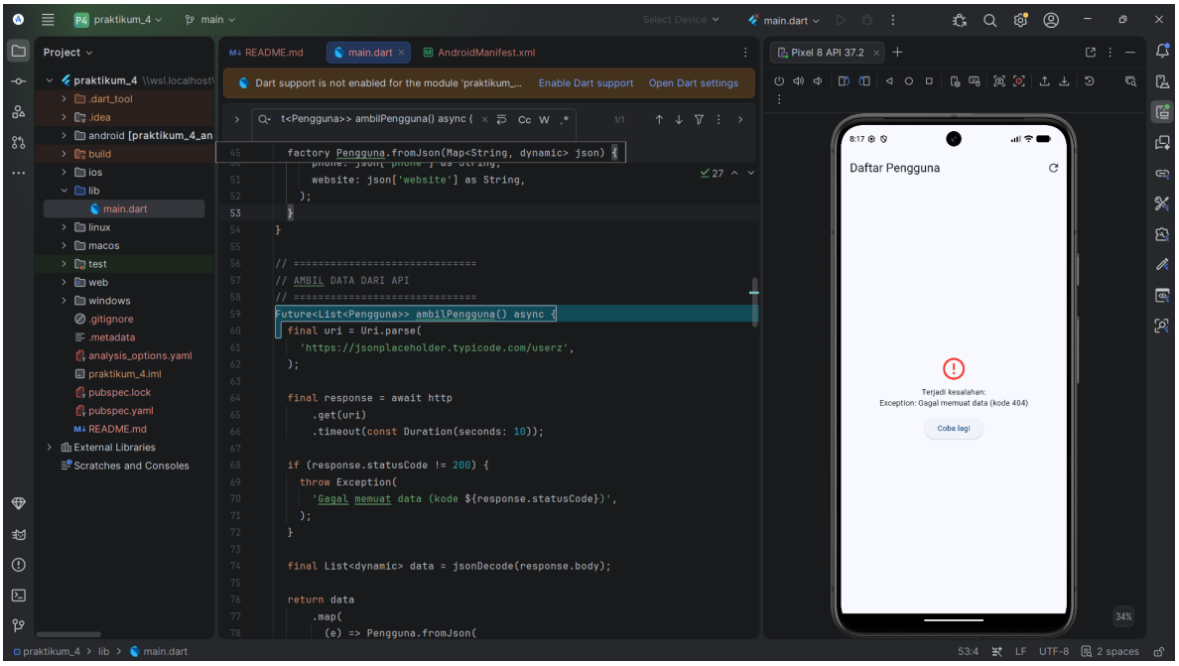
Foto 3 – Menampilkan detail pengguna



Gambar 3. Dokumentasi hasil praktikum – Menampilkan detail pengguna

Foto ketiga menunjukkan halaman detail pengguna setelah salah satu item pada daftar dipilih. Pengguna yang ditampilkan adalah Leanne Graham, dengan informasi seperti email, nomor telepon, dan website. Perpindahan halaman menggunakan Navigator.push() dan MaterialPageRoute, sedangkan objek Pengguna dikirim ke DetailPenggunaPage. Analisis dari tahap ini menunjukkan bahwa data yang sudah diterima dari API dapat digunakan kembali pada halaman lain melalui parameter. Pendekatan tersebut menghindari kebutuhan untuk mengambil data yang sama kembali ketika pengguna hanya ingin melihat detail.

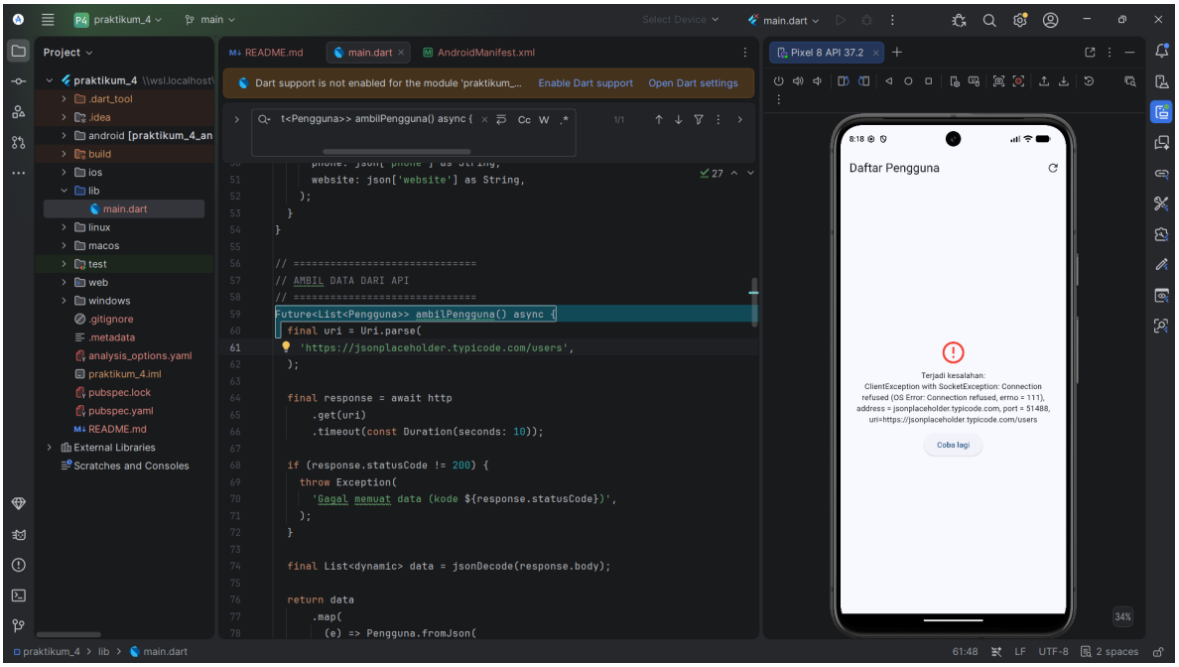
Foto 4 – Percobaan mengambil data API dengan endpoint yang salah



Gambar 4. Dokumentasi hasil praktikum – Percobaan mengambil data API dengan endpoint yang salah

Foto keempat memperlihatkan pengujian error ketika endpoint API yang digunakan adalah https://jsonplaceholder.typicode.com/user. Aplikasi menampilkan "Exception: Gagal memuat data (kode 404)". Status 404 berarti resource yang diminta tidak ditemukan. Dari sisi program, kondisi tersebut ditangani melalui pemeriksaan response.statusCode != 200. Jika status bukan 200, aplikasi tidak melanjutkan proses normal dan menghasilkan exception. Foto ini membuktikan bahwa mekanisme error handling bekerja ketika URL API tidak sesuai.

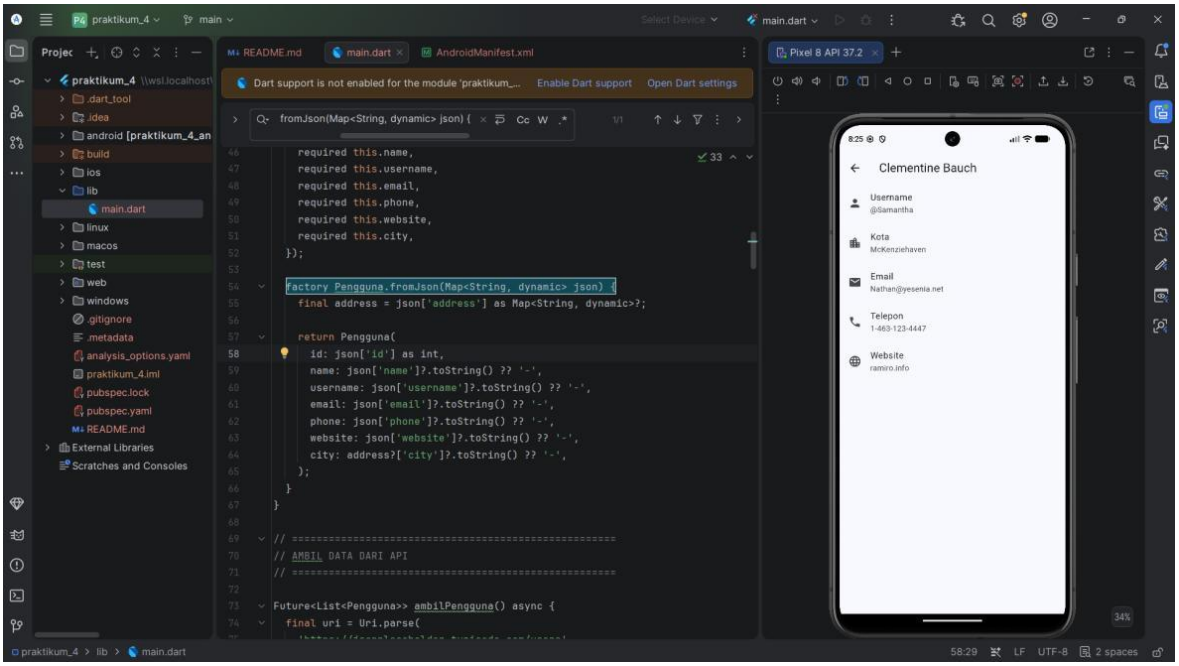
Foto 5 – Perbaikan endpoint tetapi terjadi masalah koneksi



Gambar 5. Dokumentasi hasil praktikum – Perbaikan endpoint tetapi terjadi masalah koneksi

Foto kelima memperlihatkan endpoint yang sudah diperbaiki menjadi https://jsonplaceholder.typicode.com/users. Namun, aplikasi masih gagal mengambil data dan menampilkan "ClientException with SocketException: Connection refused". Kondisi ini berbeda dari Foto 4 karena masalahnya bukan lagi endpoint 404, melainkan koneksi menuju server API. Dokumentasi juga menunjukkan penggunaan timeout selama 10 detik pada http.get(). Hal tersebut penting agar request memiliki batas waktu dan aplikasi tidak terus menunggu ketika koneksi tidak tersedia.

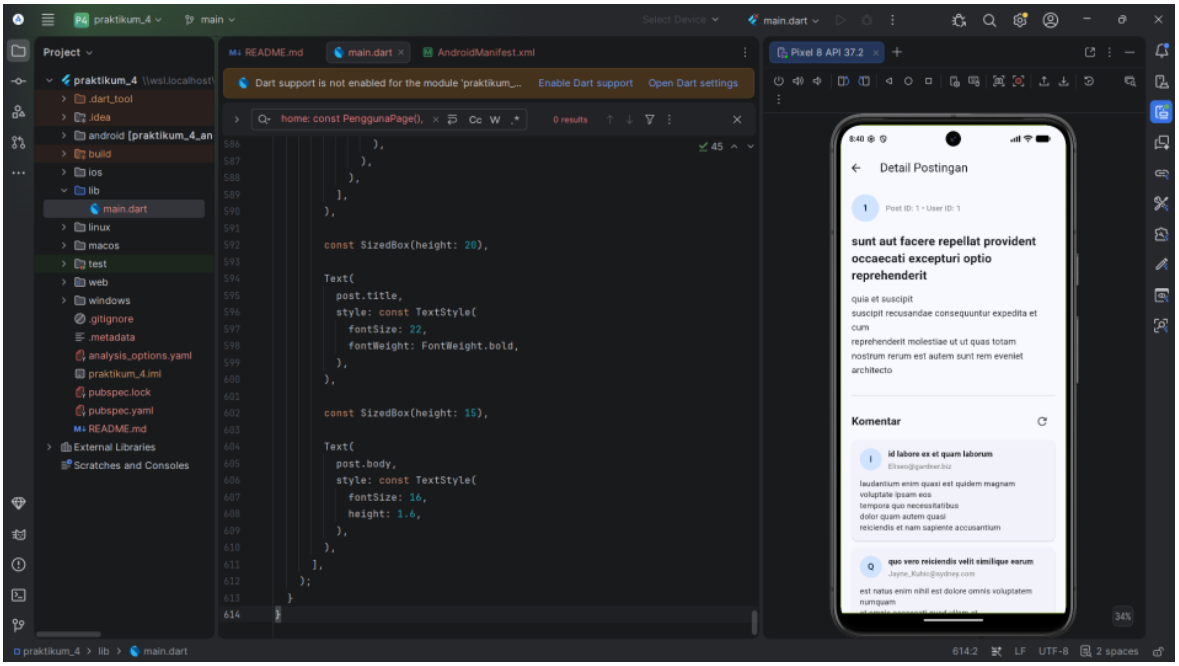
Foto 6 – Konversi data JSON menjadi objek Pengguna



Gambar 6. Dokumentasi hasil praktikum – Konversi data JSON menjadi objek Pengguna

Foto keenam memperlihatkan penerapan factory Pengguna.fromJson(Map<String, dynamic> json) untuk mengubah data JSON menjadi objek Pengguna. Field yang diproses mencakup id, name, username, email, phone, website, dan city. Pada emulator terlihat detail pengguna Clementine Bauch dengan informasi username, kota, email, telepon, dan website. Tahap ini menunjukkan bahwa data API telah berhasil direpresentasikan sebagai objek Dart yang terstruktur. Penggunaan model membuat pengelolaan data menjadi lebih rapi dan memudahkan pengiriman data dari satu halaman ke halaman lain.

Foto 7 – Menampilkan detail postingan dan komentar



Gambar 7. Dokumentasi hasil praktikum – Menampilkan detail postingan dan komentar

Foto ketujuh memperlihatkan halaman “Detail Postingan” yang berisi Post ID, User ID, judul, dan isi postingan. Pada bagian bawah terdapat bagian “Komentar” yang menampilkan komentar pengguna lain beserta nama dan alamat email. Kode menggunakan Text() untuk menampilkan informasi dan SizedBox untuk mengatur jarak antar elemen. Foto ini menunjukkan tahap pengolahan data yang lebih kompleks karena aplikasi menangani informasi postingan sekaligus komentar yang berkaitan dengan data pengguna. Dengan demikian, aplikasi sudah memperlihatkan alur pengolahan data API yang lebih luas daripada sekadar daftar pengguna.

BAB V – HASIL PENGUJIAN DAN PEMECAHAN MASALAH

Berdasarkan dokumentasi, pengujian dilakukan tidak hanya pada kondisi berhasil tetapi juga pada kondisi gagal. Pengujian pertama menunjukkan kondisi endpoint yang salah dengan respons 404. Hal ini membuktikan bahwa pengecekan status response diperlukan sebelum data diproses. Setelah URL diperbaiki, pengujian berikutnya menghasilkan Connection refused. Perbedaan kedua kondisi tersebut penting untuk dipahami karena

penanganannya berada pada level yang berbeda: 404 berkaitan dengan resource atau URL yang diminta, sedangkan Connection refused menunjukkan kegagalan koneksi ke server.

Jika aplikasi menampilkan error 404, langkah pemeriksaan yang dilakukan adalah mengecek kembali endpoint yang digunakan dan memastikan path sesuai dengan API yang dituju. Dokumentasi menunjukkan perubahan dari /user menjadi /users. Jika aplikasi menampilkan Connection refused, pemeriksaan perlu diarahkan pada koneksi jaringan, akses server API, dan kondisi emulator atau lingkungan tempat aplikasi dijalankan. Penggunaan timeout juga perlu dipertahankan agar request tidak menunggu tanpa batas.

Setelah kondisi error ditangani, pengujian dilanjutkan dengan memastikan daftar pengguna dapat tampil, item dapat dipilih, halaman detail dapat dibuka, dan data JSON dapat ditampilkan melalui objek Pengguna. Dokumentasi kemudian memperlihatkan detail postingan dan komentar sebagai hasil pengembangan lanjutan. Dengan urutan tersebut, proses pengujian dilakukan secara bertahap dari fitur sederhana menuju fitur yang memiliki data lebih kompleks.

BAB VI – ANALISIS HASIL PRAKTIKUM

Hasil praktikum menunjukkan bahwa Future dan asynchronous programming merupakan bagian penting dalam aplikasi Flutter ketika terdapat proses yang membutuhkan waktu. Contoh pada ambilSalam() memperlihatkan bagaimana Future.delayed() digunakan untuk mensimulasikan proses asynchronous. Konsep yang sama menjadi relevan ketika aplikasi menunggu respons dari API, karena request jaringan tidak dapat dianggap sebagai proses yang langsung selesai.

Penggunaan HTTP API memperlihatkan bagaimana aplikasi mobile dapat mengambil data dari sumber eksternal. Data yang diterima dalam format JSON kemudian perlu dipetakan ke struktur aplikasi melalui model Pengguna. Factory fromJson membuat proses tersebut lebih terorganisasi karena setiap field JSON memiliki tempat yang jelas dalam objek Dart. Hasilnya, UI dapat menggunakan properti objek dengan cara yang lebih mudah dipahami.

Navigasi dengan Navigator.push() dan MaterialPageRoute menunjukkan bahwa data dapat digunakan lintas halaman. Pada saat pengguna memilih salah satu item, aplikasi membawa objek Pengguna menuju halaman detail. Pendekatan ini membuat alur aplikasi

menjadi lebih natural karena pengguna dapat berpindah dari informasi ringkas menuju informasi lengkap.

Error handling menjadi bagian penting karena aplikasi yang menggunakan API bergantung pada kondisi eksternal. Dokumentasi memperlihatkan dua jenis masalah, yaitu 404 dan Connection refused. Hal tersebut menunjukkan bahwa aplikasi harus mampu membedakan kegagalan respons HTTP dengan kegagalan koneksi. Pengecekan status code dan timeout merupakan bagian dari mekanisme yang terlihat pada implementasi.

Secara keseluruhan, rangkaian dokumentasi memperlihatkan perkembangan dari aplikasi sederhana yang menghasilkan teks asynchronous menjadi aplikasi yang dapat mengambil, mengubah, menampilkan, dan menavigasikan data API. Tahap detail postingan dan komentar memperlihatkan bahwa konsep yang sama dapat dikembangkan untuk data yang memiliki hubungan lebih kompleks.

BAB VII – KESIMPULAN

Berdasarkan seluruh proses dan dokumentasi, praktikum ini berhasil memperlihatkan penerapan konsep Future dan asynchronous programming, komunikasi HTTP API, parsing JSON menggunakan model fromJson, navigasi antarhalaman menggunakan Navigator, serta error handling untuk kondisi API dan jaringan. Aplikasi dapat menampilkan daftar pengguna, detail pengguna, serta detail postingan dan komentar secara dinamis.

Dari sisi pembelajaran, praktikum ini menunjukkan bahwa pengembangan aplikasi mobile membutuhkan penggabungan antara antarmuka, proses asynchronous, struktur model data, komunikasi dengan sumber eksternal, navigasi, dan penanganan kesalahan. Tahapan yang ditunjukkan pada screenshot dapat dijadikan alur pengerjaan: memastikan project berjalan, menguji Future, menambahkan request API, memetakan JSON ke model, menampilkan daftar, membuat detail, menguji error, dan kemudian mengembangkan tampilan data yang lebih kompleks.

LAMPIRAN – URUTAN SINGKAT PENGERJAAN

Urutan pengerjaan praktikum dapat diringkas sebagai berikut. Pertama, buka project praktikum_4 dan pastikan main.dart dapat dijalankan pada emulator. Kedua, buat fungsi asynchronous ambilSalam() dan pastikan hasil “Halo dari masa depan!” muncul. Ketiga, siapkan package http dan dart:convert untuk komunikasi API serta parsing JSON. Keempat, lakukan request ke endpoint /users dan periksa response.statusCode. Kelima, buat model Pengguna dan factory fromJson untuk memetakan field JSON. Keenam, tampilkan daftar pengguna menggunakan ListTile. Ketujuh, tambahkan Navigator.push() menuju halaman DetailPenggunaPage. Kedelapan, lakukan pengujian endpoint salah untuk memastikan error 404 tertangani. Kesembilan, uji kondisi koneksi dan timeout. Kesepuluh, tampilkan detail postingan dan komentar sebagai pengembangan data yang lebih kompleks.